基于 YOLO 的车牌号识别系统学习文档

一．环境搭建：

1.从哔哩哔哩搜索yolo11安装视频及安装文档，进行认真学习，并按照视频进行了安装，由于初学，安装均不成功，通过多次摸索，最终主要参考以下链接完成：

`https://www.bilibili.com/opus/1021487181719404550`，YOLOv11(Ultralytics)环境配置，适合0基础纯小白，手把手教 - 哔哩哔哩

Anaconda3文件包下载网站：`https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D`

Pycharm安装包下载

`https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows`

两个文件选择合适版本进行安装。中间由于版本问题，进行了多次下载安装。

2.创建yolo11环境

由于我所使用的电脑为Intel核显，没有独立显卡无法使用GPU进行训练，需用CPU进行训练。

打开anaconda prompt，运行以下命令创建yolo11环境：

```
conda create -n yolov11 python=3.10
```

创建完成后，输入

```
conda activate yolov11
```

进入yolo环境。

3. 安装pytorch

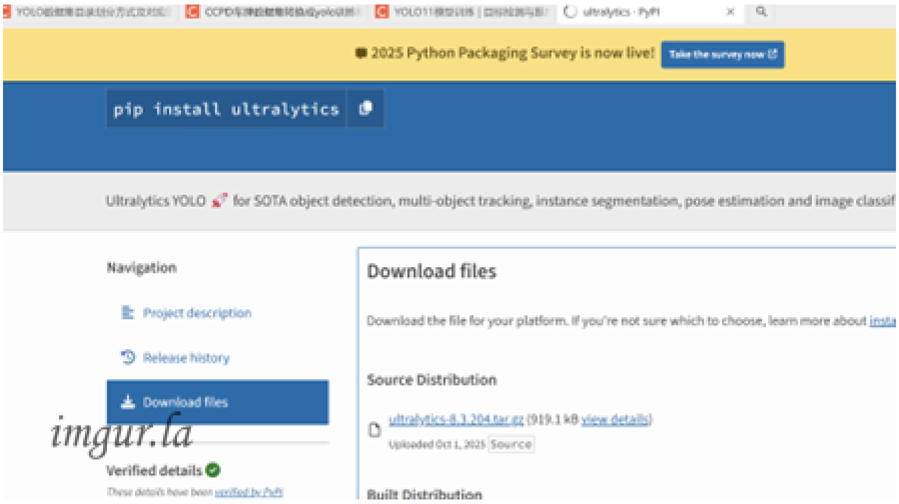
```
conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

4. 安装ultralytics库

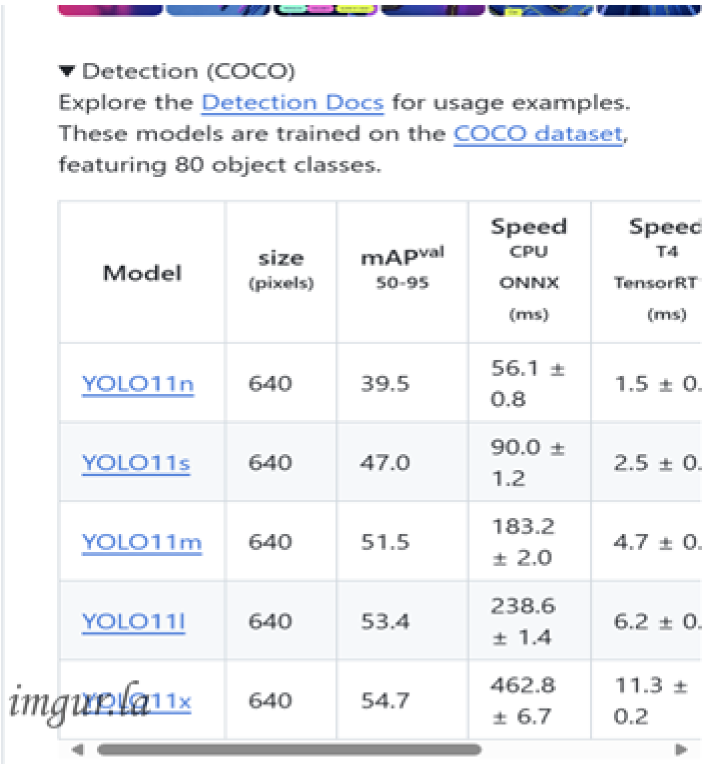
```
pip install ultralytics
```

5.下载YOLOv11（ultralytics）源码，

由于哔哩哔哩搜索的学习文档中给出的网址一直无法登录，重新学习实验室下发的链接中文档，找到下载链接，成功下载ultralytics源码见下图。



下载预训练权重文件yolo11n、yolo11s、yolo11m，下载界面如下：



▼ Detection (COCO)
Explore the Detection Docs for usage examples.
These models are trained on the COCO dataset, featuring 80 object classes.

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT (ms)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2

6. pycharm导入环境

导入界面：

二．下载和标注数集

1.按照文档内容，下载数据集。

网址：`https://pan.baidu.com/s/1-WmsMt7Zzx3jmLHVEYpdaw`

提取码: 2111

2.学习文档中ccpd数据集处理，并从哔哩哔哩搜索LabelStudio安装教程及数据标记教程。

参考文档及视频：`https://blog.csdn.net/m0_72915515/article/details/134248502#:~:text=%E9%87%87%E7%94%A8Anaconda%E5%88%9B%E5%BB%BA%E5%90%8D%E4%B8%BAlabel_sudio%E7%9A%84%E8%99%9A%E6%8B%9F%E7%8E%AF%E5%A2%83`，标注工具——Label Studio安装与简单使用-CSDN博客

`https://www.bilibili.com/video/BV1Ht4y1d7Yr/?spm_id_from=333.337.search-card.all.click&vd_source=0e4eaed4406fd009f7a2bd20660a23a4`，数据标注的方法 制作自己的数据 labelstudio的使用_哔哩哔哩_bilibili

`https://www.bilibili.com/video/BV1K3zwYQEgT/?spm_id_from=333.337.search-card.all.click&vd_source=0e4eaed4406fd009f7a2bd20660a23a4`， label studio使用教程_哔哩哔哩_bilibili

打开anaconda prompt，运行以下命令：

```
pip install Label-Studio
```

安装成功后，运行Label-Studio start，进入web页面进行数据集标注。

3.在导入本地图片过程中，由于使用了中文翻译版，一直提示“要插入新节点的节点不是该节点的子节点”错误

查找多个文档未解决，最后重新**登陆英文版界面**，成功将数据导入，并完成标注，进行导出，一共完成113张车牌数据标注。

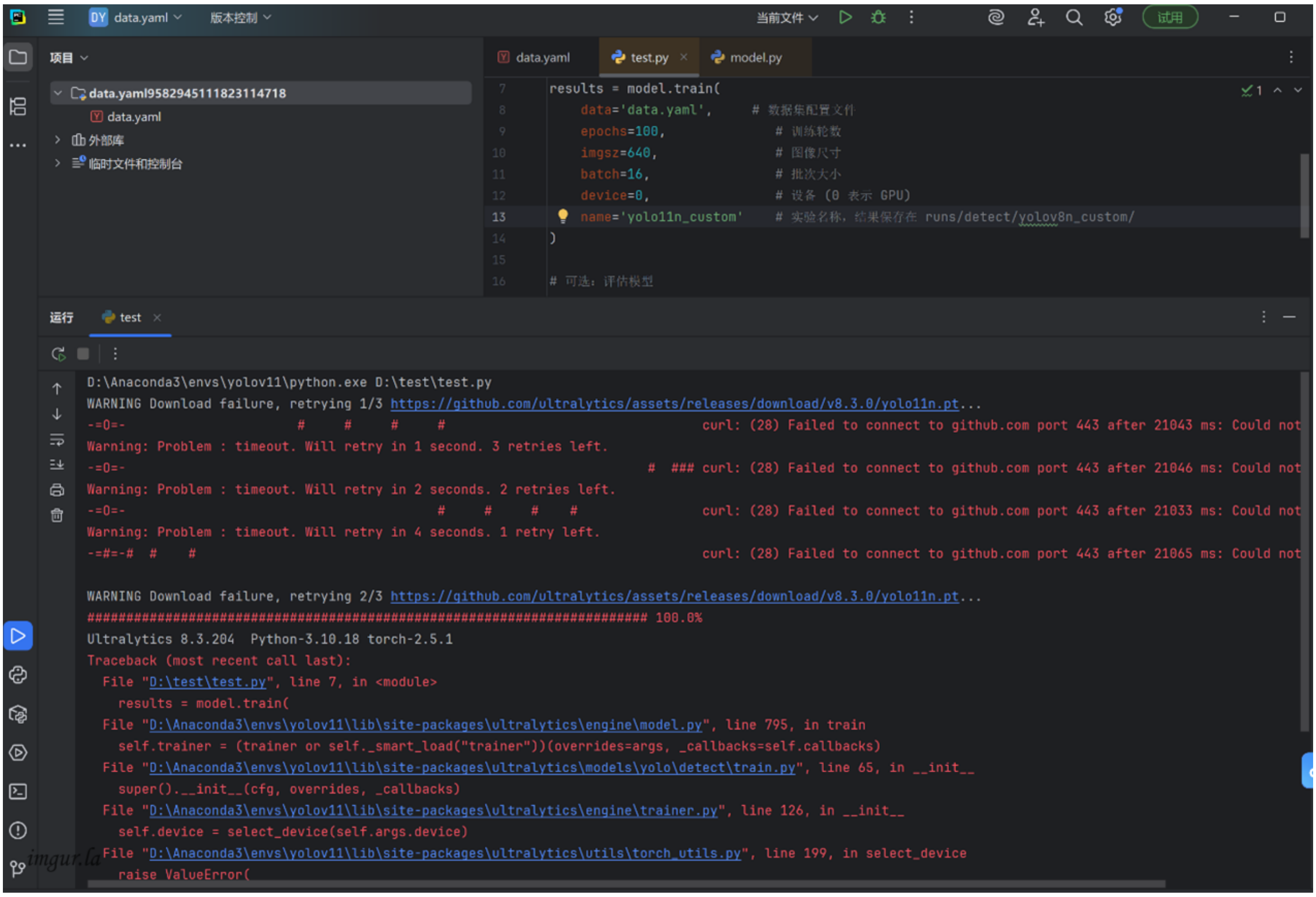
三、划分数据集，配置并训练模型

1.数据集划分：

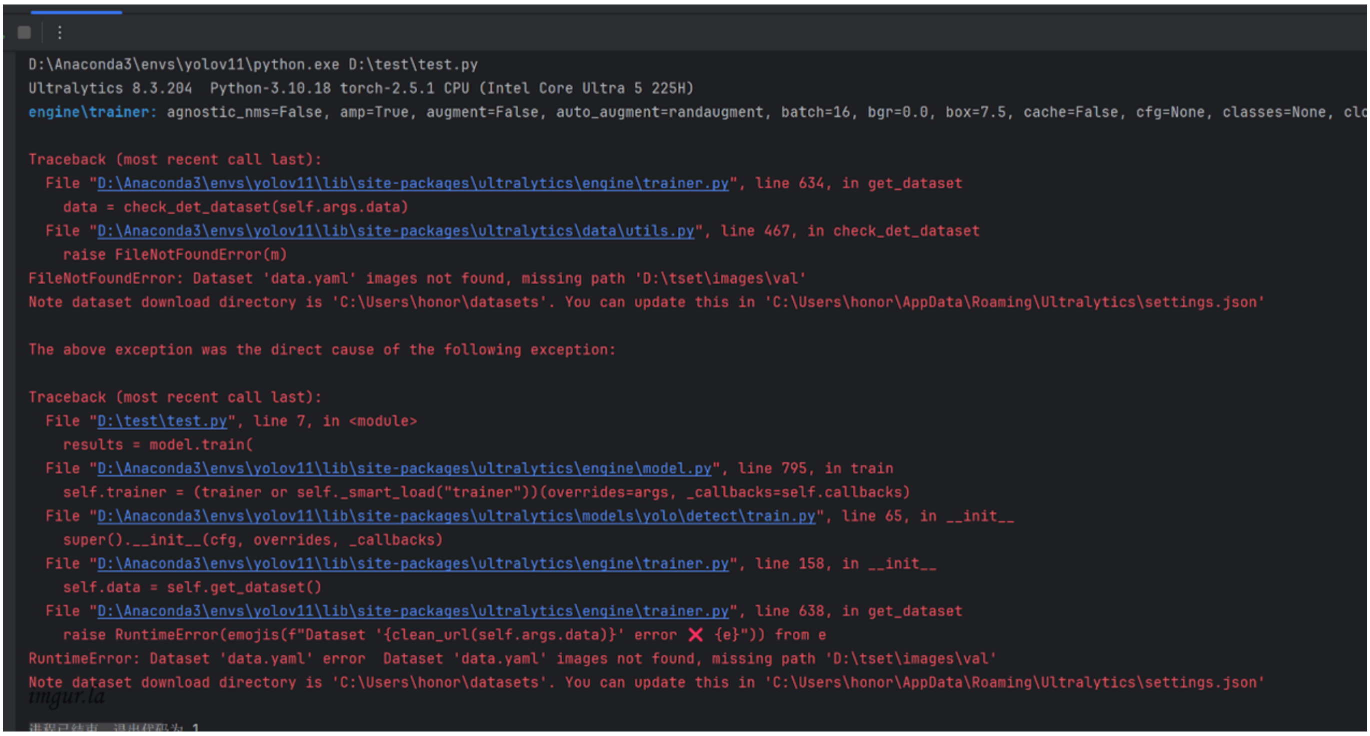
根据文档，将数据集目录在划分为train、val、test三个目录。

2.data.yaml文档配置及模型训练

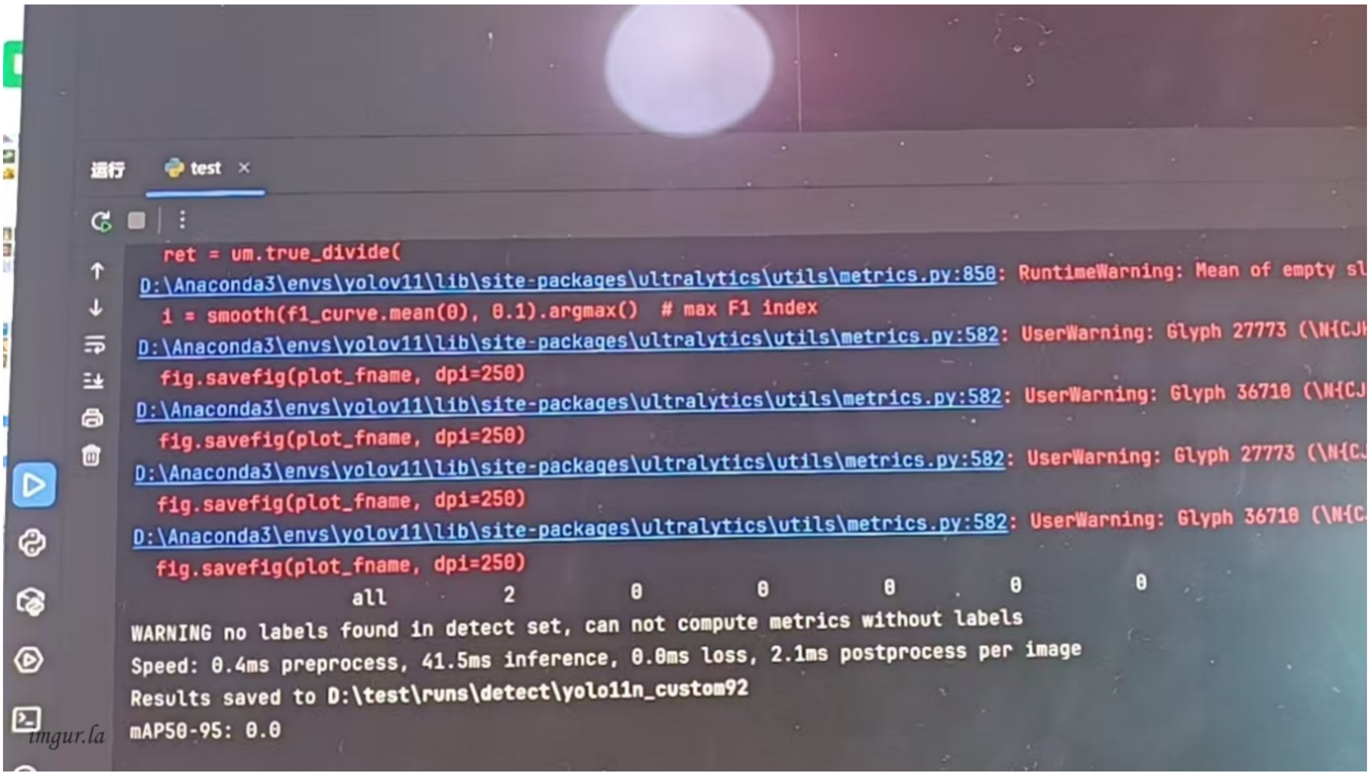
第一便训练提示如下错误，根据错误提示先进行网络配置排查，最终**调整网络设置**进行解决：



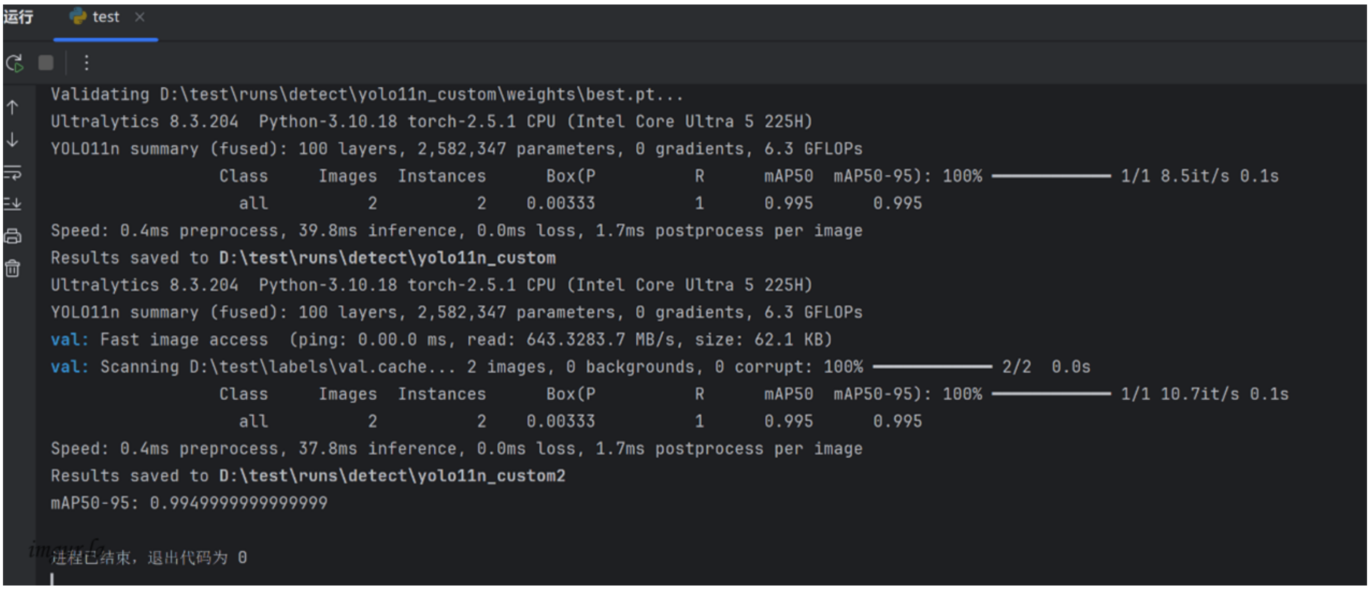
第二遍训练提示如下错误，经过检查问data.yaml文件配置中**文件名输入错误**导致：



第三遍训练一直如下错误，经过多次多次排查，最终发现**标签txt文件导出为空**导致，重新使用label studio进行标注后导出文件，为便于排查，本次只使用6张车牌，结果成功导出：



所有文件修改后进行再次进行训练，成功：



```
Validating D:\test\runs\detect\yolo11n_custom\weights\best.pt...
Ultralytics 8.3.204  Python-3.10.18 torch-2.5.1 CPU (Intel Core Ultra 5 225H)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━ 1/1 8.5it/s 0.1s
                   all          2          2    0.00333          1      0.995      0.995
Speed: 0.4ms preprocess, 39.8ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to D:\test\runs\detect\yolo11n_custom
Ultralytics 8.3.204  Python-3.10.18 torch-2.5.1 CPU (Intel Core Ultra 5 225H)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access  (ping: 0.00.0 ms, read: 643.3283.7 MB/s, size: 62.1 KB)
val: Scanning D:\test\labels\val.cache... 2 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━ 2/2  0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━ 1/1 10.7it/s 0.1s
                   all          2          2    0.00333          1      0.995      0.995
Speed: 0.4ms preprocess, 37.8ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to D:\test\runs\detect\yolo11n_custom2
mAP50-95: 0.9949999999999999

进程已结束，退出代码为 0
```

Data.yaml文档代码

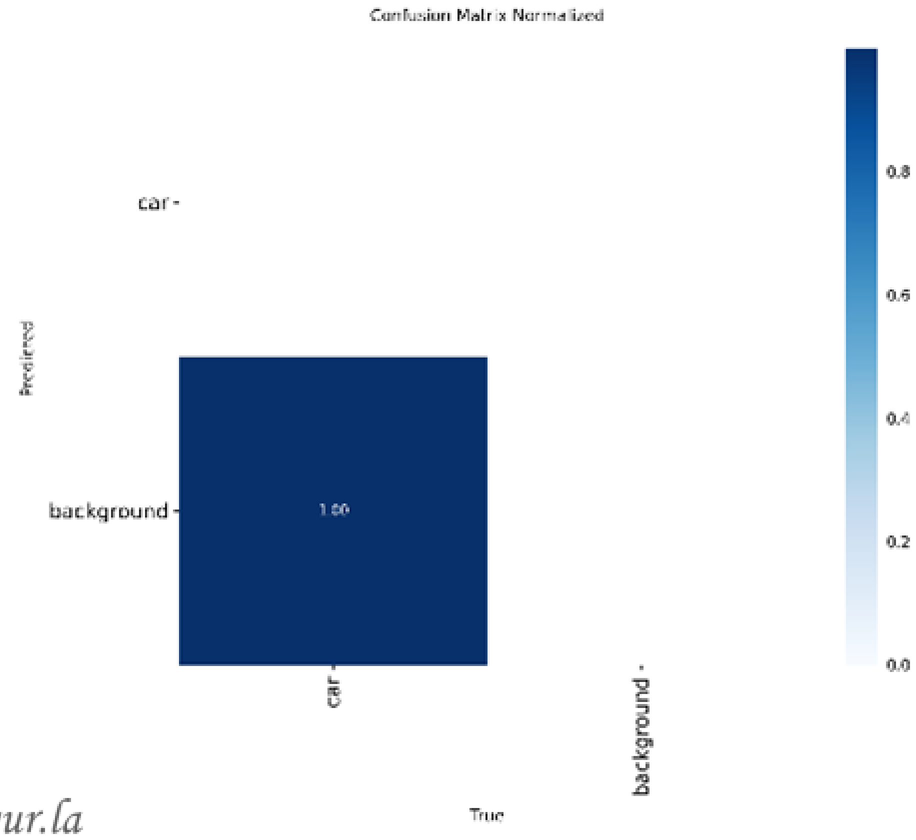
```yaml
path: d:/test
train: images/train #训练集
val: images/val #验证集
test: images/test #测试集
nc: 1
names: ['car']
```

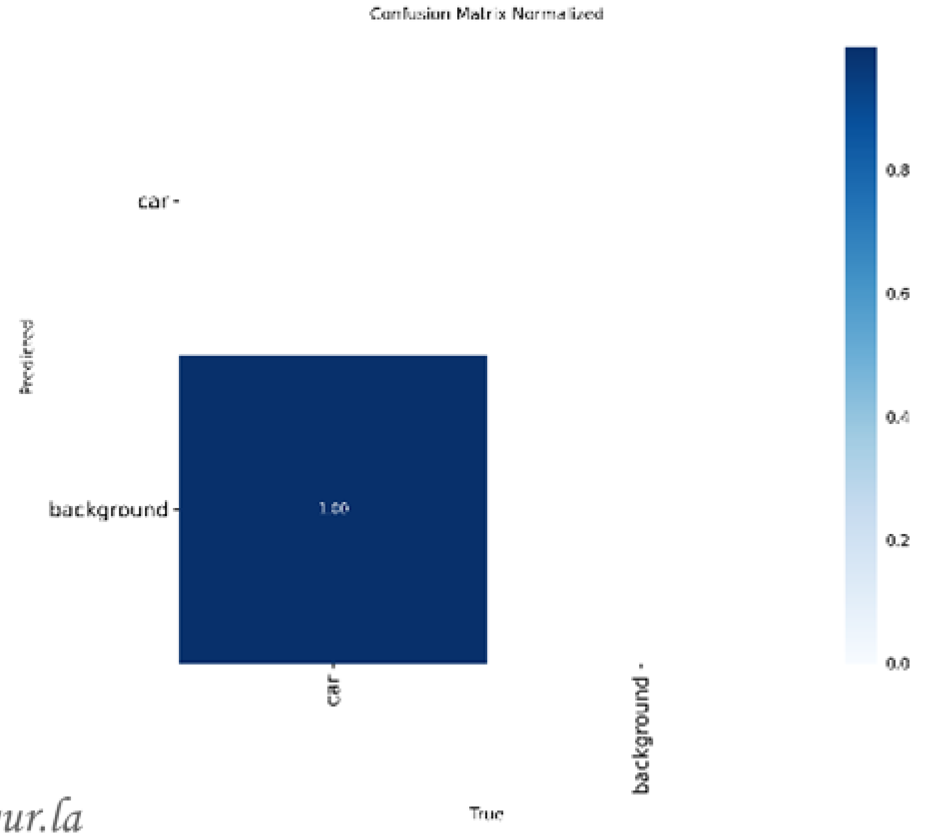
test.py程序代码：

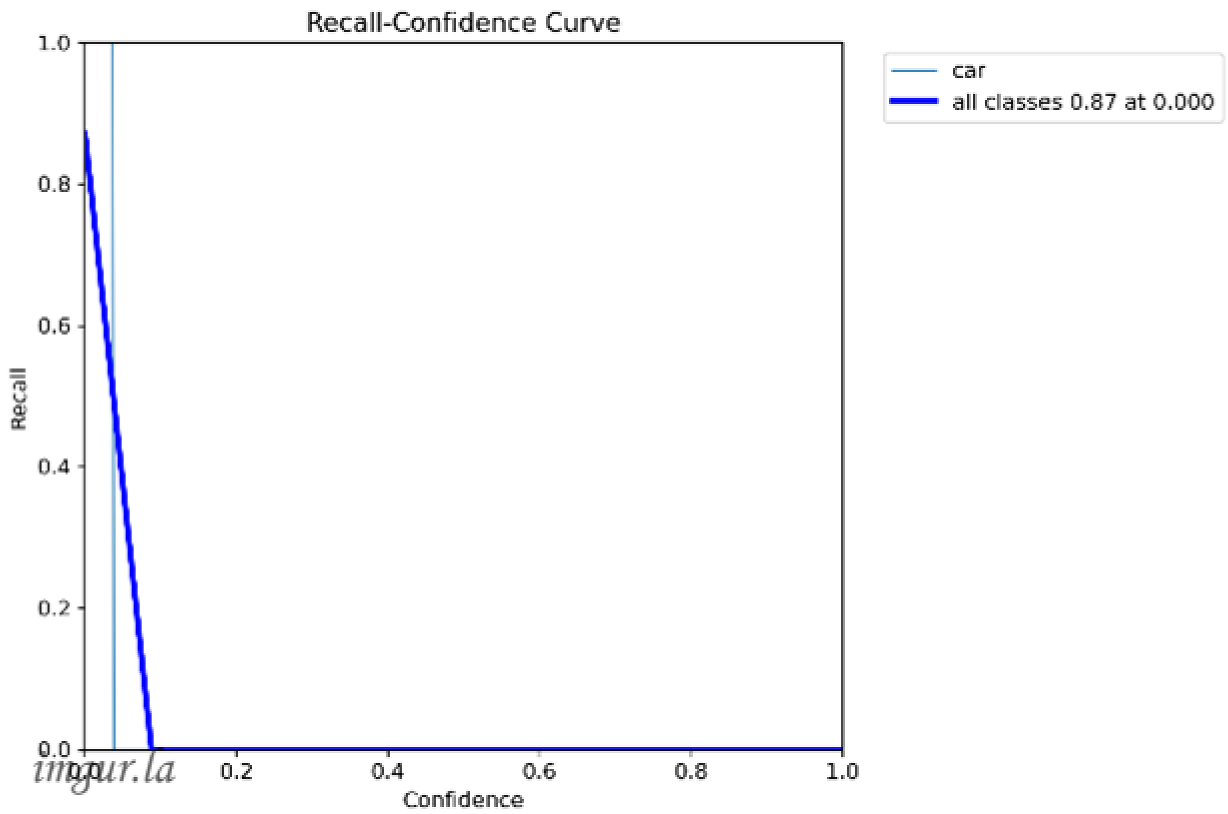
```python
from ultralytics import YOLO

# 加载 YOLOv11n 模型
model = YOLO('yolo11n.pt')  # 使用预训练模型

# 开始训练
results = model.train(
    data='data.yaml',     # 数据集配置文件
    epochs=100,            # 训练轮数
    imgsz=640,             # 图像尺寸
    batch=16,              # 批次大小
    device='cpu',              # 设备使用cpu
    name='yolo11n_custom'    # 实验名称，结果保存在 runs/detect/yolo11n_custom/
)

# 可选：评估模型
metrics = model.val()
print("mAP50-95:", metrics.box.map)
```

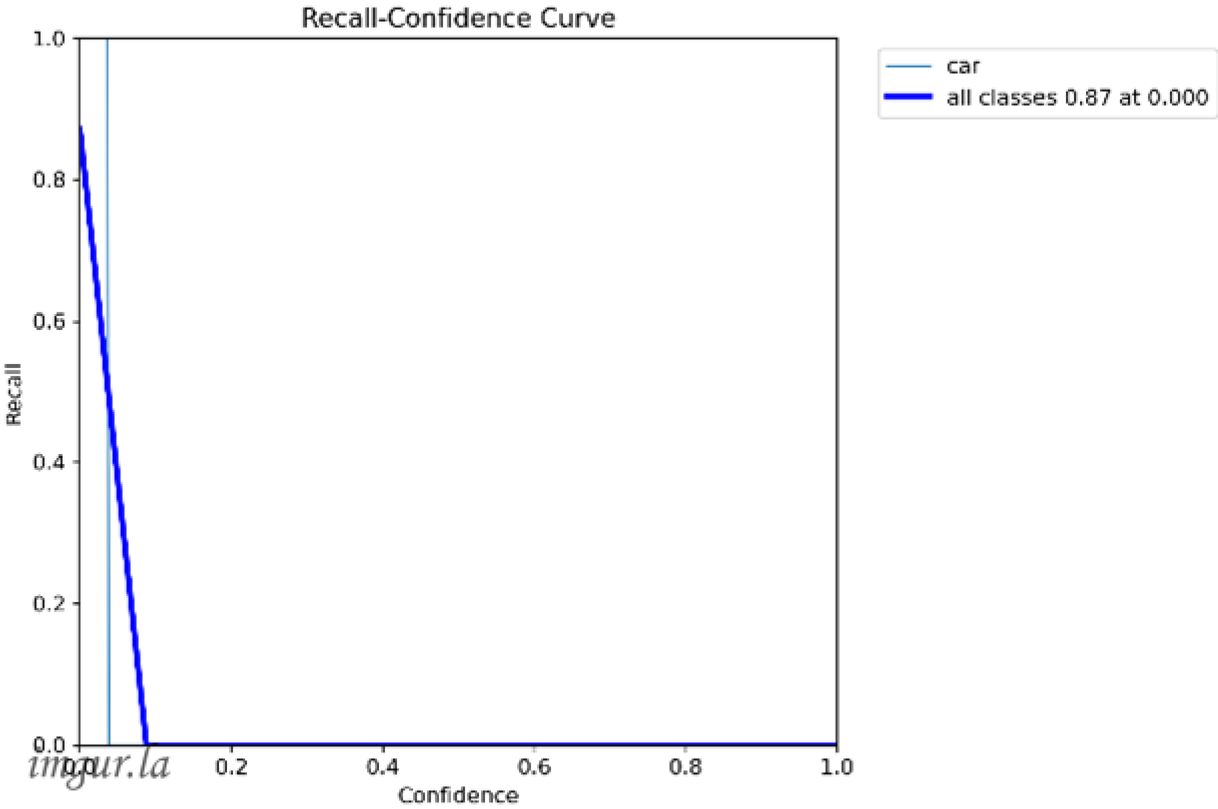
3.训练结束输出结果：

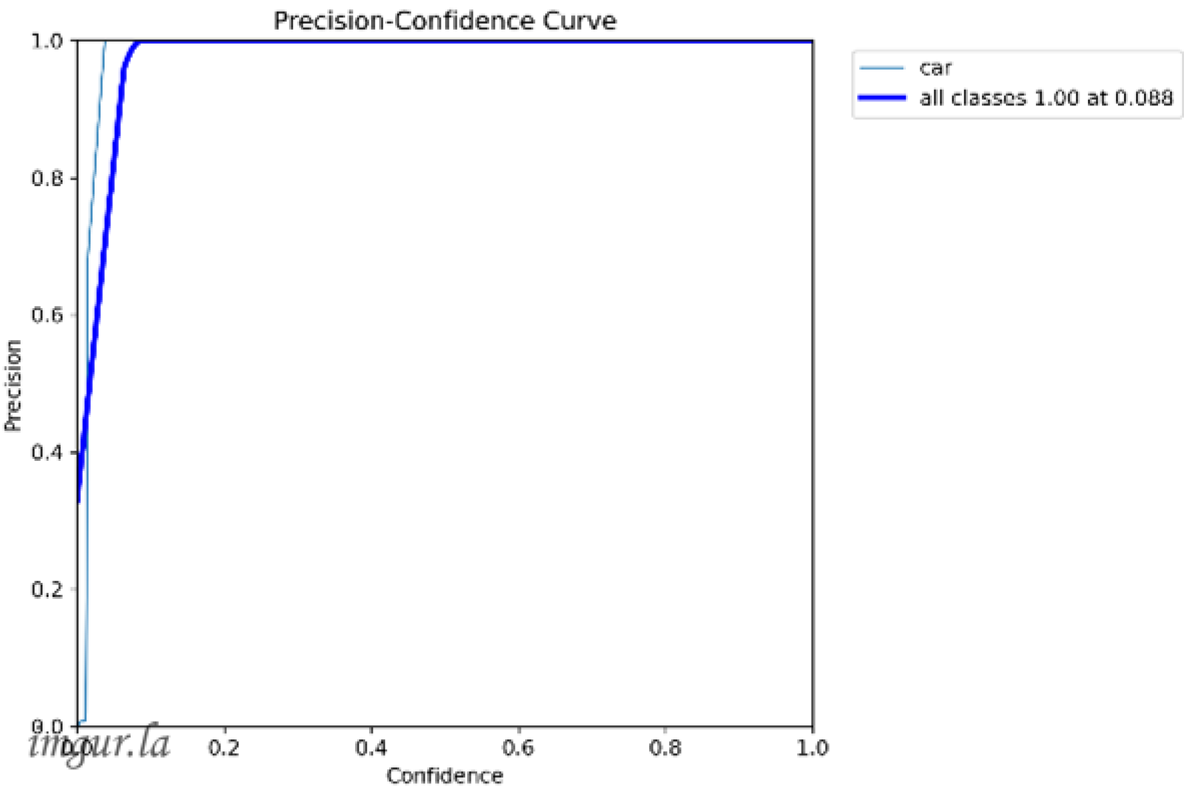

car
car

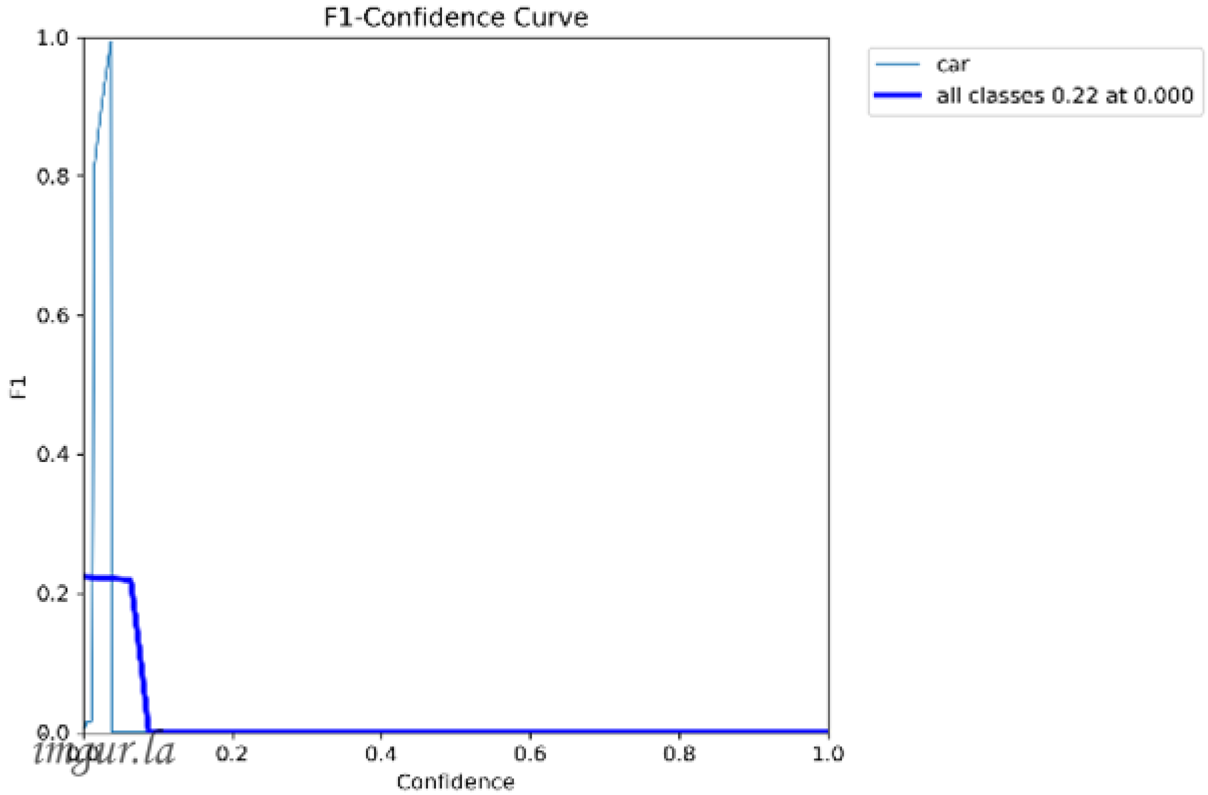

Confusion Matrix Normalized
car
background
Predicted
1.00
car
background
True
0.8
0.6
0.4
0.2
0.0


Confusion Matrix Normalized
car
background
Predicted
1.00
car
background
True
0.8
0.6
0.4
0.2
0.0


Recall-Confidence Curve
car
all classes 0.87 at 0.000
Recall
Confidence
0.0
0.2
0.4
0.6
0.8
1.0


Recall-Confidence Curve
1.0
0.8
0.6
0.4
0.2
0.0
Recall
0.2
0.4
0.6
0.8
1.0
Confidence
car
all classes 0.87 at 0.000
imgur.la

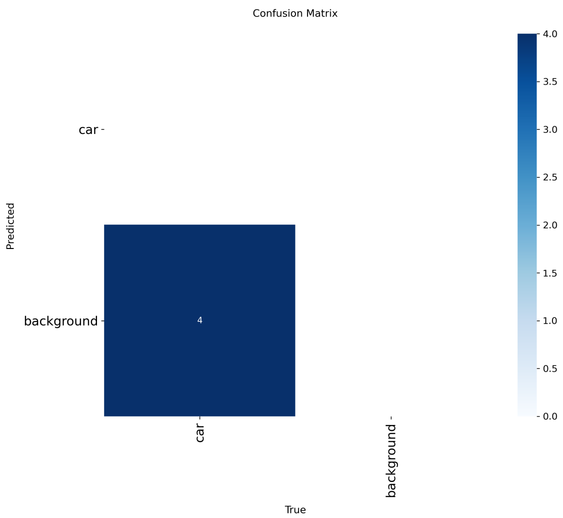

Precision-Confidence Curve
1.0
0.8
0.6
0.4
0.2
0.0
Precision
0.2
0.4
0.6
0.8
1.0
Confidence
car
all classes 1.00 at 0.088
imgur.la

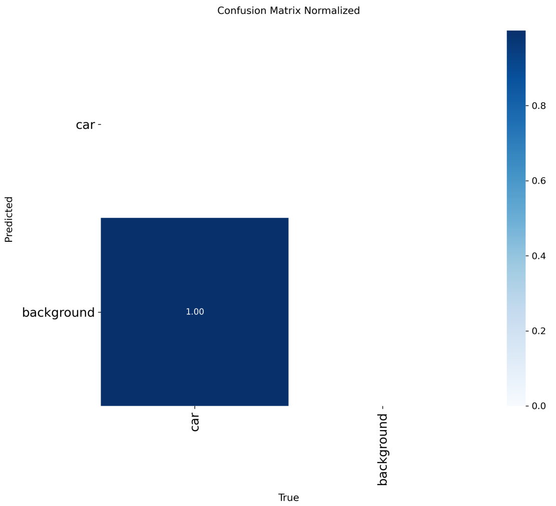


4.从网上随机下载图片，按照上述流程重新进行数据集标注进行训练（下载8张图片进行训练），具体流程跟3步骤相同。

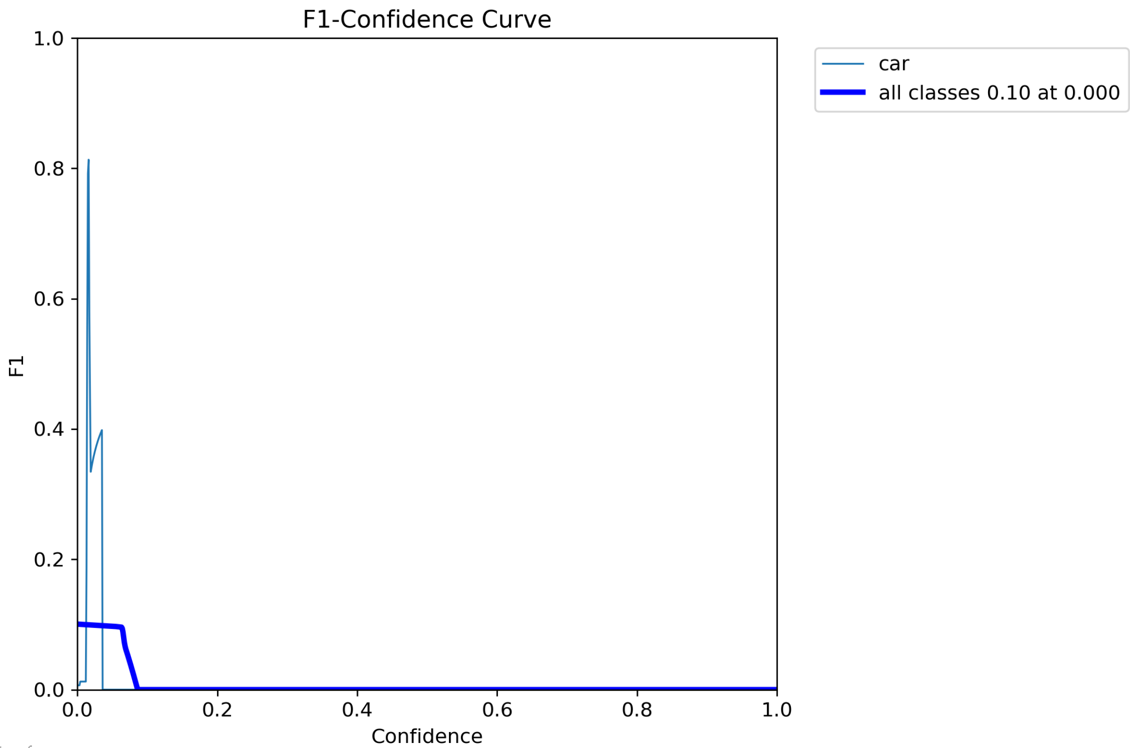

8.jpg
6.jpg
5.jpg
7.jpg


8.jpg
6.jpg
5.jpg
7.jpg
car
car
car
car

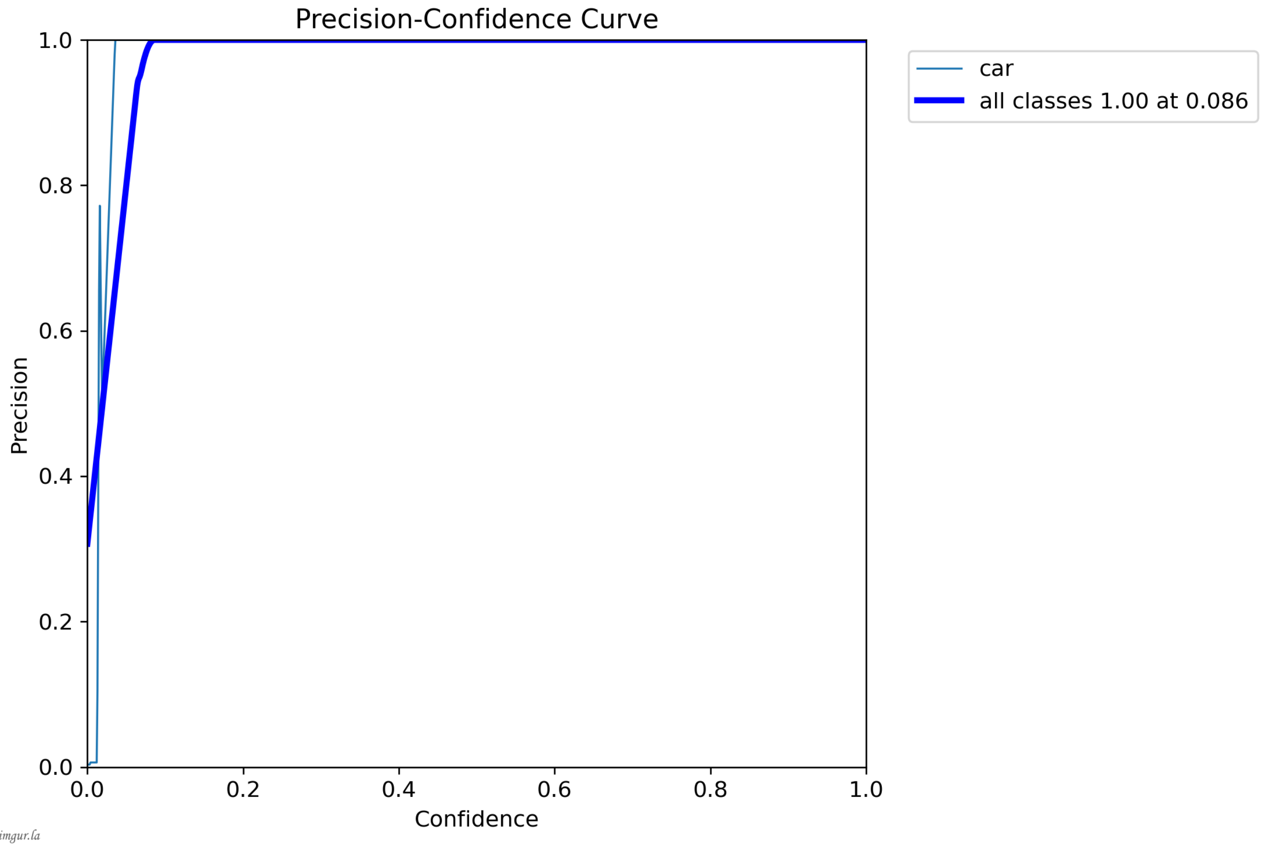

Confusion Matrix
car
background
Predicted
4
car
background
True
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0

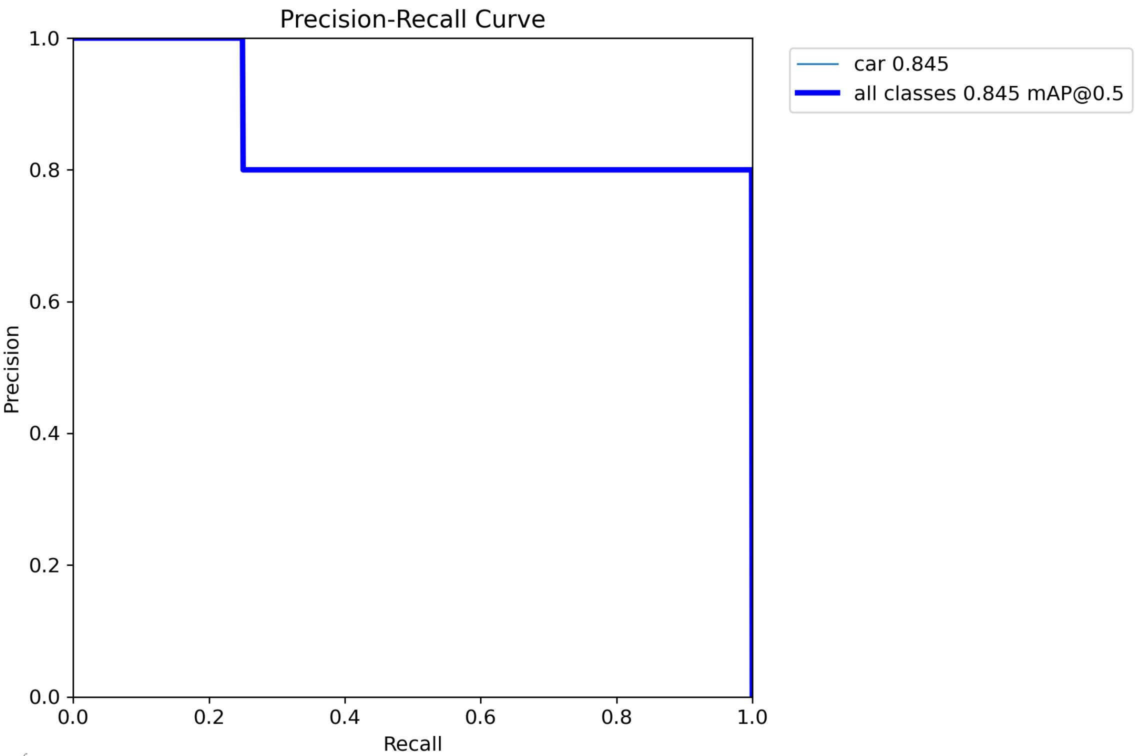


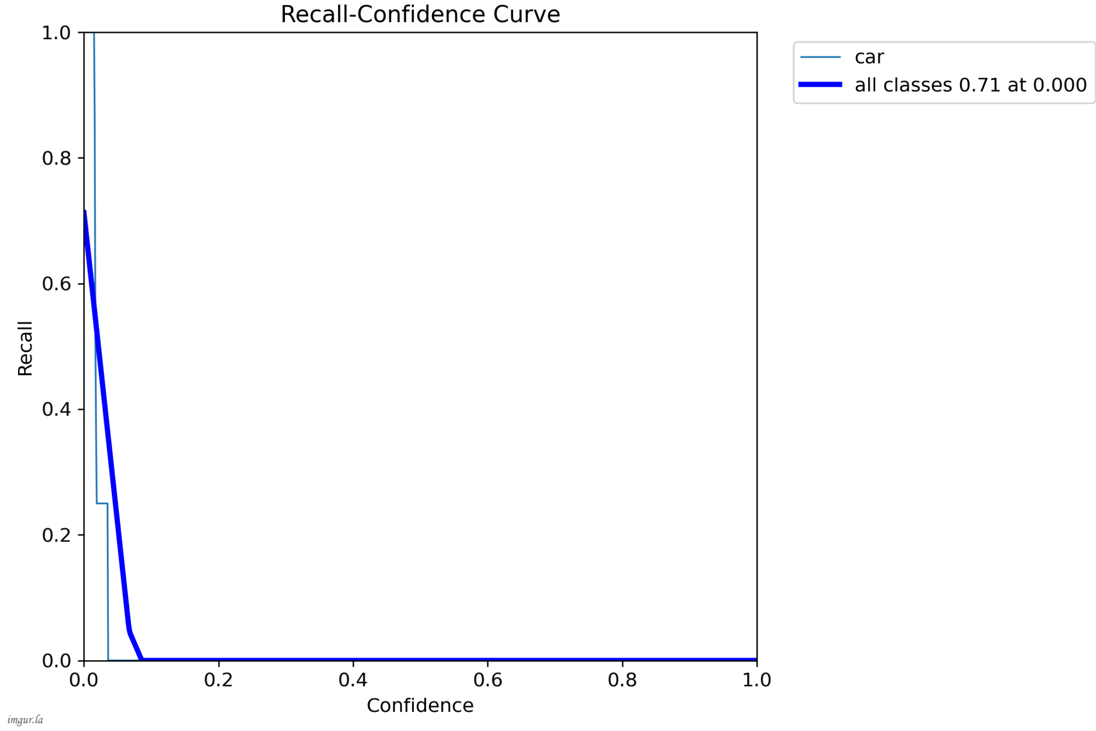
imgur.la



imgur.la


Precision-Confidence Curve
car
all classes 1.00 at 0.086
Precision
Confidence
0.0
0.2
0.4
0.6
0.8
1.0


Precision-Recall Curve
car 0.845
all classes 0.845 mAP@0.5
Precision
Recall
0.0
0.2
0.4
0.6
0.8
1.0



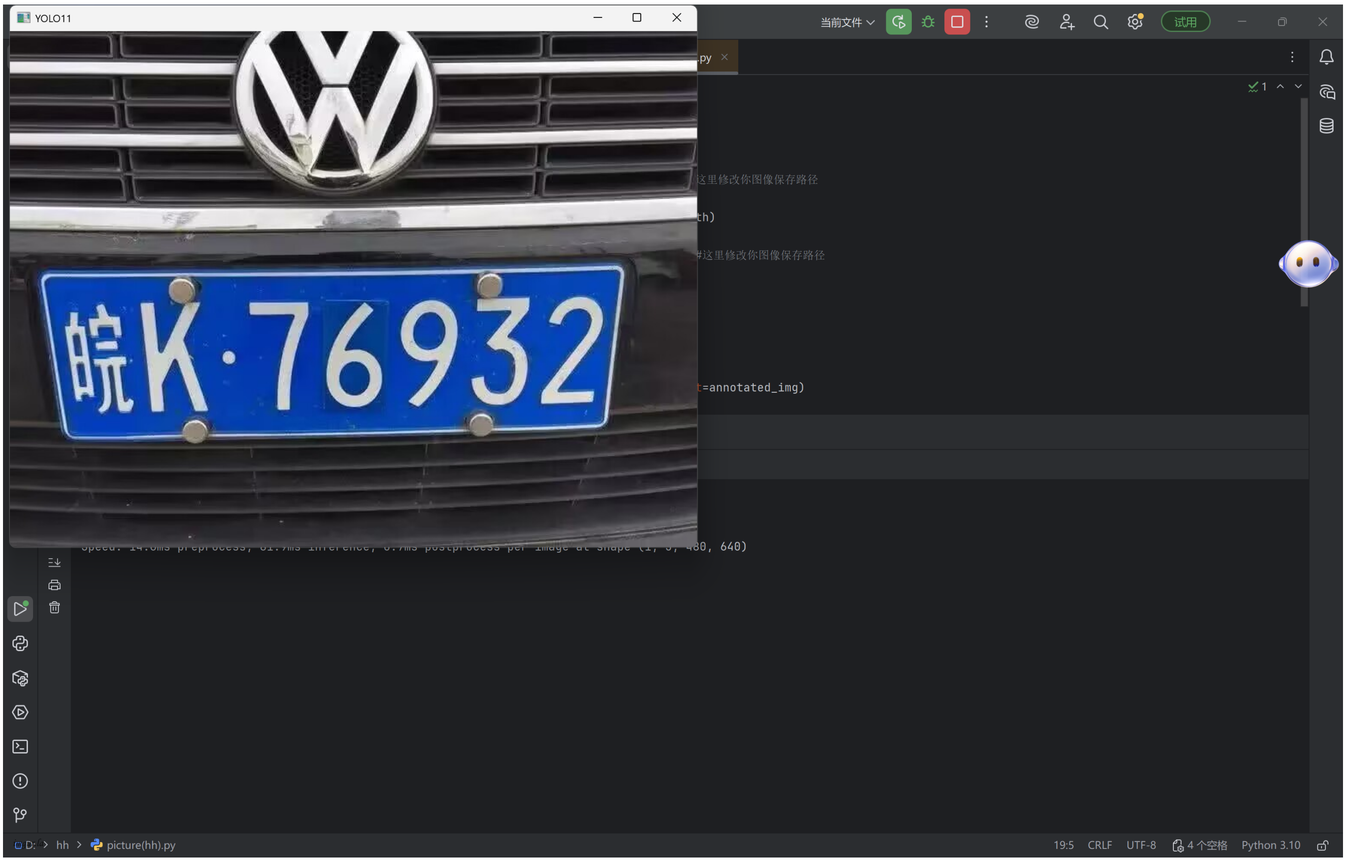
四、使用训练好的模型进行推理（图片及视频均为网上随机下载）

1.图片推理

新建xunlian程序，代码如下（从文档中复制后修改为实际文件路径等参数）：

```
import cv2
# 引入YOLO模型
from ultralytics import YOLO
# 打开图像
img_path = "d:/xunlian/img.jpg"  # 这里修改你图像保存路径
# 打开图像
img = cv2.imread(filename=img_path)
# 加载模型
model = YOLO(model="yolo11n.pt")  # 这里修改你图像保存路径
# 正向推理
res = model(img)
# 绘制推理结果
annotated_img = res[0].plot()
# 显示图像
cv2.imshow(winname="YOLO11", mat=annotated_img)
# 等待时间
cv2.waitKey(delay=10000)
# 绘制推理结果
cv2.imwrite(filename="jieguo.jpeg", img=annotated_img)
```

显示结果如下，图片成功。

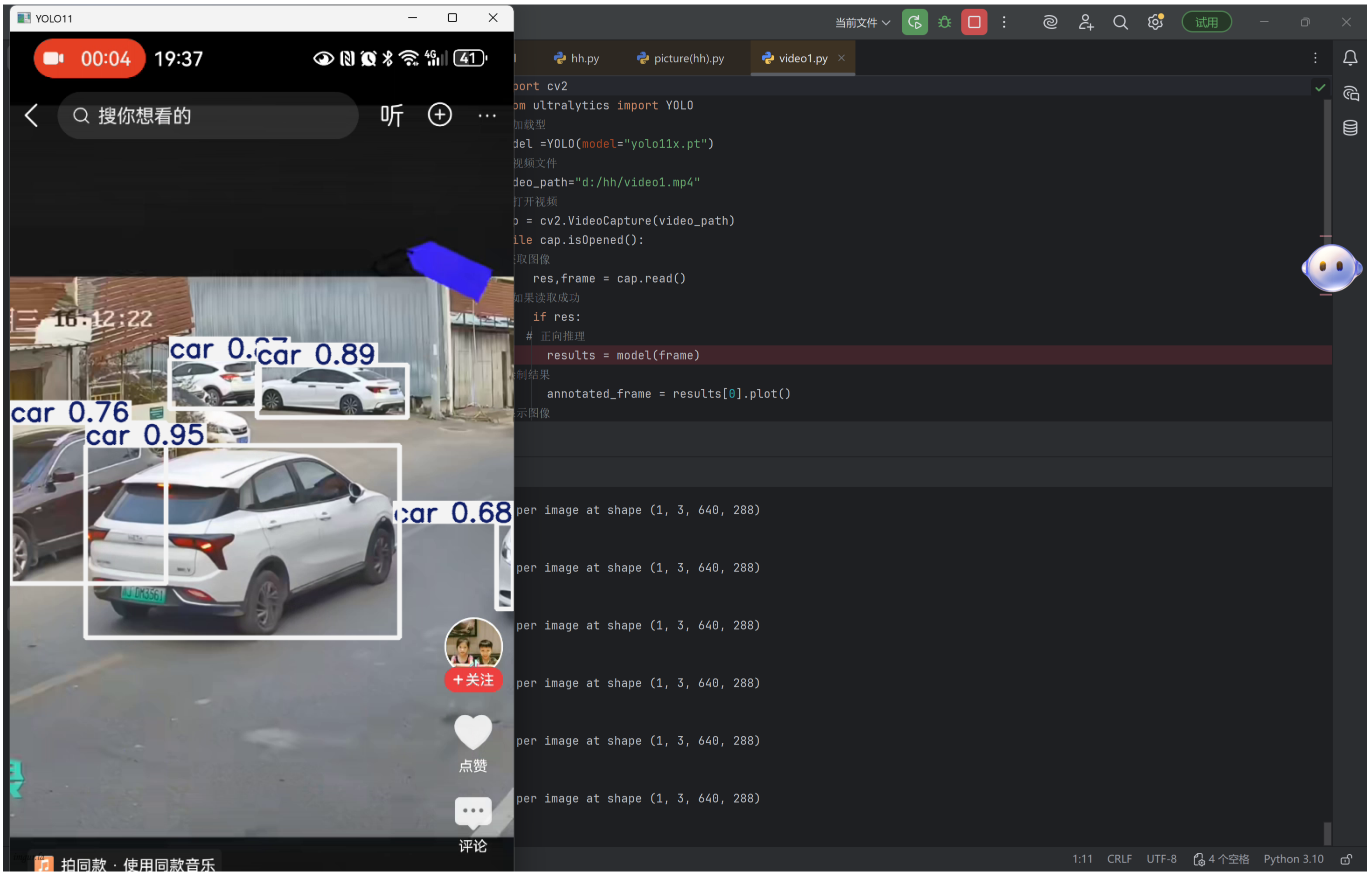


2.视频推理

新建shipinxunlian程序，代码如下（从文档中复制后修改为实际文件路径等参数）：

```python
import cv2
from ultralytics import YOLO
# 加载模型
model = YOLO(model="yolo11x.pt")
# 视频文件
video_path = "d:/xunlian/chepai.mp4"
# 打开视频
cap = cv2.VideoCapture(video_path)
while cap.isOpened():
    # 获取图像
    res, frame = cap.read()
    # 如果读取成功
    if res:
        # 正向推理
        results = model(frame)
        # 绘制结果
        annotated_frame = results[0].plot()
        # 显示图像
        cv2.imshow(winname="YOLO11", mat=annotated_frame)
        # 按ESC退出
        if cv2.waitKey(1) == 27:
            break
    else:
        break
# 释放链接
cap.release()
# 销毁所有窗口
cv2.destroyAllWindows()
```

显示结果如下，视频成功



YOLO11
00:04 19:37
搜你想看的
听
16:12:22
car 0.89
car 0.76
car 0.95
car 0.68
+关注
点赞
评论
拍同款 · 使用同款音乐
当前文件
试用
hh.py
picture(hh).py
video1.py
port cv2
om ultralytics import YOLO
加载型
del =YOLO(model="yolo11x.pt")
视频文件
deo_path="d:/hh/video1.mp4"
打开视频
p = cv2.VideoCapture(video_path)
ile cap.isOpened():
取图像
res,frame = cap.read()
如果读取成功
if res:
# 正向推理
results = model(frame)
制结果
annotated_frame = results[0].plot()
示图像
per image at shape (1, 3, 640, 288)
per image at shape (1, 3, 640, 288)
per image at shape (1, 3, 640, 288)
per image at shape (1, 3, 640, 288)
per image at shape (1, 3, 640, 288)
per image at shape (1, 3, 640, 288)
1:11 CRLF UTF-8 4 个空格 Python 3.10